VIKA Documentation

Release 0.1.0

Elias Bitsch & Viktoriia Ovdiienko

Jun 22, 2026

Table of Contents

Two cooperating mobile brick-laying robots for the FHTW MRE2 Robotermodellierung course.

Chapter 1

Introduction

VIKA (Virtual Industrial Kinematic Arm) is a simulated modular masonry automation cell developed for the FHTW course MRE2 Robotermodellierung (SS2026). Two self-designed, cooperating robots build a brick wall autonomously inside a Gazebo construction site, and the whole cell is operated from a browser-based Human-Machine Interface (HMI).

1.1 Motivation

The construction industry faces a persistent shortage of skilled masons, while bricklaying remains physically demanding, repetitive and injury-prone. Automated masonry is therefore an active field of industrial development. VIKA is inspired by two real products:

- FBR Hadrian X, a truck-mounted boom that lays blocks from a CAD model.
- Construction Robotics SAM100, a semi-automated mason that places bricks and applies mortar alongside a human crew.

VIKA reproduces the core idea of these systems, a coordinated pick, place and bond cycle driven from a digital model of the wall, at simulation scale and with two cooperating arms instead of one.

1.2 Project goals

The project is built around the assignment requirements and adds several of the optional features:

1. Two self-designed serial kinematics, one per team member, with self-designed end effectors.
2. A cooperative industrial use case with genuine interaction between the two robots.
3. A simulation in which both robots are visible and moving, plus an HMI that moves a tool centre point (TCP) linearly using inverse kinematics.
4. A single start script and this PDF documentation.
5. As a bonus, a web GUI, plus the groundwork for an AI layer (local-LLM voice control and an open-vocabulary brick detector) documented as future work.

1.3 Team and roles

Author	Robot	Role
Elias Bitsch	VIKA, a 6-DOF articulated arm with a vacuum suction gripper	Pick and place. Picks a row of bricks from the pallet and sets them on the wall (6-DOF inverse kinematics).
Viktoriia Ovdiienko	VIKA 5, a 5-DOF arm with a cement nozzle	Line application. Traces the wall top edge as a Cartesian path and extrudes a cement bead.

The two kinematics are deliberately different (6 versus 5 degrees of freedom, and fundamentally different end effectors), so that each team member contributes one distinct, self-designed robot, as required by the assignment.

1.4 Document structure

This documentation follows the chapters required by the assignment: the use case, the robot construction, the simulation, the HMI and the AI modules, followed by installation and launch instructions and an API reference.

Chapter 2

Industrial Use Case

2.1 Scenario: Modular Masonry Automation Cell

Two autonomous robots build a brick wall in a construction cell. Each robot sits on its own linear rail so that it can traverse the full length of the wall. The two rails run parallel to each other on opposite sides of the wall line, and the robots face each other across it.

- robot_a (VIKA, Elias) is a 6-DOF articulated arm with a vacuum suction gripper. Its three suction pads pick a row of three bricks at once from the pallet and place them on the wall.
- robot_b (VIKA 5, Viktoriia) is a 5-DOF arm with a cement nozzle. It traces the current top edge of the wall as a Cartesian path and extrudes a cement bead onto which the next course is bonded.

2.2 Cooperative build cycle

The two robots share one process, which is the core requirement of the assignment, by alternating between bonding and placing for every course of the wall:

1. Reset and supply. A pallet of bricks sits at the edge of the cell, and the dynamic pick row (three fused bricks) is dropped onto the pallet.
2. Bond (robot_b). VIKA 5 traverses its rail and traces the current wall top edge with the cement nozzle, laying a continuous cement bead.
3. Pick (robot_a). VIKA lowers its suction gripper onto a brick row, attaches the three bricks and lifts them.
4. Place (robot_a). VIKA carries the row to the wall and sets it on the fresh cement bead, then releases the vacuum.
5. Bond next course (robot_b). The nozzle applies cement on top of the new course.
6. Loop. Steps 3 to 5 repeat course by course until the wall is complete.

Both robots run on rails along the same wall and depend on each other's output: there is no placing without a cement bead, and no next bead without a placed course. The interaction is therefore a genuine shared process rather than two independent tasks.

2.3 Industrial context and economic rationale

The scenario has a clear industrial and economic justification:

- Labour shortage. Skilled masons are increasingly scarce, and automation keeps output up without proportionally scaling the crew.
- Throughput and consistency. A robotic cell lays courses at a constant rate with repeatable joint geometry and bead thickness.
- Safety. Heavy, repetitive lifting is moved off the human worker. In a real deployment the cell would be fenced and interlocked.
- Driven by a digital model. The wall geometry comes from a CAD or HMI model, so the same cell can build any wall layout without re-tooling, mirroring the CAD-to-build workflow of the Hadrian X.

2.4 Real-world references

- FBR Hadrian X, an automated block-laying boom.
- Construction Robotics SAM100, a semi-automated mason and mortar applicator.

Robot Construction

Both robots are modelled in xacro (parameterised URDF) under `vika_description/urdf`. Each robot is a purely serial kinematic chain mounted on a world-anchored linear rail. The two arms are self-designed: their dimensions are sized to a realistic industrial class (link lengths, masses, joint limits and torques), but the geometry is original and is not a copy of any commercial robot.

3.1 Shared base: linear rail

Both robots use the same base concept (`base_rail.xacro`):

- A world-anchored rail, 12.5 m long, with each robot anchored via the `base_x` and `base_yaw` xacro arguments. robot_a sits at `base_x = -2.0` (yaw 0) and robot_b at `base_x = +0.8` (yaw pi, so it faces robot_a). The rail is world-fixed, so the spawn pose does not move it.
- A prismatic carriage that slides along the rail, driven by the `rail_controller`. This lets each arm traverse the full wall length.

The rail base replaced an earlier tracked mobile platform. It gives the same "reach the whole wall" capability while keeping the simulation numerically stable and easy to control.

3.2 robot_a (VIKA): 6-DOF articulated arm (Elias)

A classic anthropomorphic RRR-RRR arm (`arm_6dof.xacro`): a shoulder, an elbow and a 3-DOF spherical wrist. Six degrees of freedom give full position and orientation control of the tool, which is what pick-and-place onto an angled wall course needs.

Table 3.1. robot_a joint limits and torques

Joint	Function	Range	Torque
J1	base rotation	plus/minus 185 deg	80 Nm
J2	shoulder	-155 to +35 deg	80 Nm
J3	elbow	-130 to +154 deg	60 Nm
J4	wrist roll	plus/minus 350 deg	20 Nm
J5	wrist pitch	plus/minus 130 deg	20 Nm
J6	tool rotation	plus/minus 350 deg	15 Nm

- Reach: about 1.4 m. Payload: about 7 kg (a brick row plus the gripper, with reserve).

- Level of detail: the visual model uses Fusion-exported STL meshes, while the collision model uses primitive shapes (boxes and cylinders) for stable, fast physics.

3.3 robot_b (VIKA 5): 5-DOF arm (Viktoriia)

A standalone 5-DOF serial arm with its own geometry. It deliberately has one axis fewer: for tracing a cement line, rotation about the nozzle's long axis is functionless because the nozzle is rotationally symmetric, so that wrist axis is dropped (the chain is an RRR shoulder and elbow plus a 2-DOF wrist).

Table 3.2. robot_b joint limits and torques

Joint	Function	Range	Torque
J1	base rotation	plus/minus 185 deg	80 Nm
J2	shoulder	-150 to +40 deg	80 Nm
J3	elbow	-125 to +150 deg	55 Nm
J4	wrist pitch	plus/minus 130 deg	20 Nm
J5	nozzle approach angle	plus/minus 180 deg	15 Nm

- Reach: about 1.5 m, slightly longer so it can reach over the wall top edge.
- Payload: about 5 kg (nozzle plus cement reserve).
- Five degrees of freedom are sufficient because the cement bead only requires positioning the nozzle tip and its approach angle along the wall edge.

3.4 End effectors

Each tool is its own xacro and is attached at the arm flange (`*_arm_tool0`).

Vacuum suction gripper (`tool_gripper.xacro`, robot_a):

- A horizontal bar carrying three suction pads spaced one brick apart, so the gripper picks a full row of three (fused) bricks in a single motion.
- The pallet shows a flat 3x3 layer of three rows, but only `row_0_0` is *dynamic* and respawning; `row_1_0` and `row_2_0` are static decoration. Grasping is simulated with a single Gazebo `DetachableJoint` plugin bound to `row_0_0`: publishing on its attach topic fixes that whole row to the pads, and detaching releases it. `reset_bricks.sh` re-drops the dynamic row onto the pallet at startup, and `refill_pallet.sh` respawns it after each course.

Cement nozzle (`tool_cement.xacro`, robot_b):

- A nozzle head that extrudes a cement bead as the arm traces the wall edge.
- Placed bricks are bonded to the wall after a bead is laid.

3.5 Realistic design

- Masses and inertia are set per link, and joint limits and torques are sized to a realistic industrial arm class.

- Self-collision is disabled in the SRDF because the collision model is intentionally coarse (primitive shapes). This avoids false `START_STATE_IN_COLLISION` reports while keeping physics stable.
- Both arms spawn in a compact folded stow pose, so they never droop or collide at start-up.

Chapter 4

Simulation

4.1 Simulator and middleware

- Simulator: Gazebo Harmonic.
- Middleware: ROS 2 Jazzy with `ros2_control` and MoveIt 2.
- Runtime: everything runs inside the `vika_ros` Docker container, and the HMI runs in a separate `vika_hmi` container. The Gazebo server runs headless (`gz sim -r -s`), and an optional GUI client attaches for visualisation.

4.2 Construction-site world

The world (`vika_gazebo/worlds/construction_site.sdf`) holds the masonry cell:

- A ground plane and a pallet as the brick source.
- A flat 3x3 layer of three brick rows on the pallet. Only `row_0_0` is a *dynamic* pick row that the suction gripper grabs and that respawns; the other two rows (`row_1_0`, `row_2_0`) are static decoration. The dynamic row is modelled separately from the static wall bricks for performance.
- The wall zone between the two robot rails, where placed bricks (`wall_brick.sdf`) and the cement strip (`cement_strip.sdf`) accumulate.

Both robots are spawned into this one world at once, on their parallel rails, which satisfies the requirement that both robots are visible and moving in a single simulation. The scene is intentionally lean; extra industrial dressing such as a fence is a possible future addition and is not part of the current world.

4.3 Control stack

`ros2_control` is provided in simulation by `gz_ros2_control` (configured in `vika.ros2_control.xacro`). Per robot the controllers are:

- `joint_state_broadcaster`, which publishes `/joint_states`.
- `arm_controller`, a `JointTrajectoryController` for the arm joints.
- `rail_controller`, a `JointTrajectoryController` for the prismatic rail carriage.

The vacuum gripper is not a `ros2_control` controller. Grasping is done with a Gazebo `DetachableJoint` plugin toggled over the per-row `/suction/r0_0/attach` and `/suction`

`/r0_0/detach` topics (the row id is `r`-prefixed because a topic segment may not start with a digit).

MoveIt 2 `move_group` runs against these live controllers. Inverse kinematics use the KDL solver. The `MoveGroup` action and the TF chain (`world` to `*_arm_tcp`) are available for planning and for the HMI linear TCP jog.

4.4 Sensors and actuators

The cell integrates several simulated sensors and actuators:

- A wrist camera on the placing arm, the image source for the planned brick detector (see AI Modules (Bonus, future work)).
- Vacuum suction (a single `DetachableJoint` plugin on the dynamic row) as the grasp actuator.
- Cement extrusion as the bonding actuator.
- Joint state and TF telemetry for the digital twin.

4.5 Numerical stability

Several deliberate choices keep the simulation stable and reproducible:

- Primitive collision instead of trimesh. This removed thousands of ODE trimesh overflows and the associated crashes.
- World-anchored rails, which keep each robot's base fixed and well-conditioned.
- A folded stow pose at spawn, which avoids start-up droop or self-contact.

4.6 Verification

A test suite (`scripts/run_tests.sh`) backs the simulation:

- 18 unit tests. The xacro for both robots parses to valid URDF with the expected joints, links and limits, and the SRDF, kinematics and controller configs are well-formed and mutually consistent. These are pure parsing tests: fast, deterministic and with no running sim needed.
- 11 end-to-end smoke checks against a running sim. Both robots publish `joint_states` (arm joints and rail), values are finite, arms stay within limits (no collapse), MoveIt IK is available, and TF resolves `world` to `*_arm_tcp`.

Human-Machine Interface (HMI)

The HMI is a web application (Vite, React 19, TypeScript, shadcn/ui and three.js), which goes well beyond the console-application minimum of the assignment. It talks to ROS 2 over `rosbridge_server` (WebSocket port 9090) via `roslibjs`, and a `ros_gz_bridge` connects the Gazebo and ROS sides.

The HMI opens at `http://localhost:5173`.

5.1 Linear TCP jog (required feature)

The central required feature is moving a robot TCP linearly using inverse kinematics. VIKA provides this in two forms.

The console application (`vika_moveit/scripts/tcp_jog.py`) is the assignment minimum. It reads `x+`, `x−`, `y+`, `y−`, `z+` and `z−` commands and moves the selected arm TCP by a fixed step. Each command computes IK for the new Cartesian target and executes the trajectory, so the tool travels along a straight Cartesian line.

The web panel (`TcpJogPanel.tsx`) offers plus and minus X, Y and Z buttons with a step-size control. It performs the same IK-based Cartesian jog from the browser, with a live TCP pose readout from `/tf`.

Both rely on MoveIt 2 inverse kinematics, which satisfies the requirement that the HMI move a robot at the TCP linearly.

5.2 Panels

Panel	Function
3D digital twin	A three.js scene rendering both robot URDFs, animated live from `/joint_states` as a real-time digital twin of the Gazebo cell.
TCP jog	Linear Cartesian jog of the selected arm via inverse kinematics.
Joint sliders	Direct per-joint commanding for setup and debugging.
Rail panel	Moves each robot prismatic carriage along its rail.
Mission panel	Start, pause and abort the cooperative build mission and watch its state.
Teleop panel	Manual base and jog control via virtual joysticks (gamepad supported).
Sensor panel	Live wrist-camera view and other telemetry.
Wrist view	Dedicated camera feed from the placing arm. The planned brick-detection overlay attaches here (see AI Modules (Bonus, future work)).
Wall-draw panel	Draw a wall outline on a grid (touch-friendly). The line is sampled into brick positions and handed to the build mission.

Prompt card and voice	Natural-language and voice command input. The local-LLM parsing and spoken replies are planned future work (see AI Modules (Bonus, future work)).
Mission tree view	Live behaviour-tree state from `/bt/state`, drawn as a tree with per-node status colour.
Robot selector and tabs	Switch the active robot (A, B or both) for the control panels.

5.3 Voice interface (planned)

A microphone prompt card is in place for spoken commands. The intended pipeline parses them with a local Gemma model (served by Ollama) into `/hmi/*` commands and speaks replies back via Kokoro text-to-speech. This AI layer is documented as future work in AI Modules (Bonus, future work). The typed prompt input is the convenience layer available today.

5.4 Bridge architecture

```
Browser (React + three.js)
  |  WebSocket :9090 (roslibjs)
  v
rosbridge_server  ->  ROS 2 Jazzy nodes (MoveIt, ros2_control, mission)
                        ^
                        |  ros_gz_bridge
                        v
                    Gazebo Harmonic
```

Chapter 6

AI Modules (Bonus, future work)

The assignment lists AI integration only as an optional nice-to-have. VIKA goes beyond the core deliverable by laying the groundwork for three AI features: an open-vocabulary brick detector, a local-LLM voice control layer, and an MCP world-control server. These are planned future work. The building blocks and the wiring exist, but they are not part of the core, verified demo path. The base simulation, IK control and HMI run fully without any of them.

This chapter documents the foundation already in place and the direction each module is heading.

6.1 Open-vocabulary brick detection (Grounding DINO)

Status: groundwork laid, optional add-on.

The brick detector (`vika_moveit/scripts/dino_detector.py`) is built around the open-vocabulary, zero-shot model Grounding DINO (`IDEA-Research/grounding-dino-tiny`) via Hugging Face `transformers`. The model is prompt-driven, so no custom dataset or training is needed; it is queried with the text prompt `"a brick."`.

The intended flow runs on demand rather than continuously:

1. The HMI or the mission publishes `/hmi/detect` (`std_msgs/Empty`).
2. The node grabs the latest wrist-camera frame.
3. Grounding DINO returns bounding boxes. Weak, oversized and tiny boxes are filtered using a confidence threshold, area limits and NMS.
4. Each box centre is back-projected through the camera intrinsics and the live wrist-camera TF onto the brick-top plane, giving a world `(x, y, z)` pose.

Outputs:

- `/detect/image/compressed`, the camera frame with boxes drawn, for the HMI view.
- `/detect/result`, a `std_msgs/String` JSON message with the detections (`box`, `score` and `world` pose).

The node degrades gracefully: if `torch` or `transformers` are missing it stays alive and reports "offline", so the HMI does not crash.

6.2 Voice and natural-language control (local LLM)

Status: groundwork laid, optional add-on.

The HMI prompt card is designed to turn spoken or typed commands into robot actions:

1. Speech or text is sent to a local Gemma model served by Ollama (default `gemma4:12b` at `http:`

`//localhost:11434`), which returns a structured intent as JSON. A keyword parser is the fallback when Ollama is unreachable.

2. The intent is mapped onto the existing `/hmi/*` command topics (select robot, jog, home, start mission, detect, and so on).
3. VIKA speaks the result back through Kokoro text-to-speech (`kokoro-fastapi`, OpenAI-compatible endpoint, voice `af_heart`, English).

The design keeps the whole AI control loop local, with no cloud dependency, so that once completed an operator could command the cell hands-free.

6.3 MCP server (scaffold)

Status: scaffold.

The `vika_mcp` package provides a Model Context Protocol (MCP) server (`server.py`) that exposes world and mission controls (for example `list_models`, `spawn_model` and `mission_start`), so that an external MCP client such as Claude can drive the cell. The server is currently a scaffold: the tool list is defined, and the dispatch is being wired to the same `/hmi/*` and Gazebo interfaces the rest of the system already uses.

6.4 Why these modules matter

Once finished, these modules would demonstrate two uses of AI in an industrial cell. The first is supervisory control: a local LLM lets an operator command and query the cell in natural language, without touching code. The second is perception: open-vocabulary detection locates bricks in the camera image and returns world poses, which is the basis for a visual placement check.

For now they are documented as the project AI roadmap, with the camera, the detector node, the LLM and TTS wiring and the MCP scaffold already in the repository as the starting point.

Chapter 7

Installation

VIKA runs entirely in Docker, so the only host requirements are Docker and a copy of the repository. All ROS 2 Jazzy, Gazebo Harmonic, MoveIt 2 and HMI dependencies are baked into the two container images.

7.1 Prerequisites

- Docker with the Compose plugin (`docker compose`).
- For hardware-accelerated Gazebo rendering, the NVIDIA Container Toolkit (the compose file requests the `nvidia` runtime). Software rendering works as a fallback.
- An X server on the host for the optional Gazebo GUI (run `xhost +` once).
- Git, to clone the repository.

The HMI image bundles Node and the web dependencies, so no host Node install is required to run the dashboard.

7.2 Get the code

```
git clone https://github.com/eliasbitsch/MONUMENTAL.git
cd MONUMENTAL
```

7.3 Build the images

Two images are built from `docker/`:

- `vika/ros:jazzy`: ROS 2 Jazzy, Gazebo Harmonic, MoveIt 2 and the colcon workspace.
- `vika/hmi:dev`: the Vite and React web HMI.

```
docker compose -f docker/docker-compose.yml build
```

Note The brick-detection add-on (Grounding DINO) is optional. Bake it in permanently with:

```
WITH_PERCEPTION=1 docker compose -f docker/docker-compose.yml build ros
```

7.4 Third-party dependencies

Everything is pinned in the images and package manifests. The principal external components are:

- ROS 2 Jazzy, `ros2_control` and `gz_ros2_control`, and MoveIt 2.
- Gazebo Harmonic and `ros_gz_bridge`.
- `rosbridge_suite` (`rosbridge_server`) for the web HMI WebSocket.
- Vite, React 19, TypeScript, shadcn/ui, three.js and `roslibjs` for the HMI.
- A small custom Python behaviour-tree engine for the mission logic.
- Grounding DINO via Hugging Face `transformers` and `torch` (optional) for brick detection.
- Ollama (a local Gemma model) for voice and NL command parsing, and Kokoro (`kokoro-fastapi`) for text-to-speech. Both are optional.
- The MCP Python SDK for the world-control server scaffold.

7.5 Documentation tooling

To rebuild this document, install the doc requirements and a PDF backend:

```
pip install -r vika_docs/requirements.txt
pip install rinohtype          # pure-Python PDF backend, no LaTeX needed
```

See Launch Guide for how to render the PDF and run the simulation.

Chapter 8

Launch Guide

8.1 Start everything

A single script brings up the entire cell:

```
./start-docker.sh
```

This script does the following:

1. Starts the `vika_ros` and `vika_hmi` containers.
2. Launches the Gazebo Harmonic server headless, plus an optional GUI client.
3. Runs `vika_bringup full_demo.launch.py`, which starts the `ros_gz_bridge`, spawns both robots and starts all controllers.
4. Starts `rosbridge_server` on `ws://localhost:9090` for the HMI.
5. Resets the pick bricks onto the pallet (`reset_bricks.sh`) and folds both arms into their stow pose (`fold_arms.py`).

Endpoints once it is up:

- HMI at `http://localhost:5173`.
- rosbridge at `ws://localhost:9090`.
- Gazebo GUI in a native window (optional, cosmetic).

Stop the simulation processes (the containers stay up) with:

```
./start-docker.sh stop
```

8.2 Demo workflow

```
./start-docker.sh              # bring up gz, both robots, controllers and HMI
./scripts/run_tests.sh         # 18 unit and 11 e2e checks; proves the sim is
healthy
```

Then, from the HMI at `http://localhost:5173`:

1. Watch the 3D digital twin mirror both robots live.
2. Open the TCP jog panel, pick a robot, and press the X, Y or Z buttons. The tool moves linearly via inverse kinematics.
3. Use the rail panel to traverse a robot along its rail.

4. Start the cooperative build mission from the mission panel and watch the pick, place and cement cycle build the wall.
5. Try the wall-draw panel: sketch a wall and have the robots build it.

8.3 Console TCP jog (assignment minimum)

The required linear-TCP-jog feature is also available as a console application:

```
# once move_group is up:
ros2 launch vika_moveit robot_a_move_group.launch.py
python3 /ws/src/vika_moveit/scripts/tcp_jog.py
#   commands: x+  x-  y+  y-  z+  z-
```

8.4 Running the tests

```
./scripts/run_tests.sh unit     # 18 unit tests (xacro, SRDF, config); no sim
needed
./scripts/run_tests.sh e2e      # 11 e2e smoke checks against a running sim
./scripts/run_tests.sh          # both
```

8.5 Rebuilding this PDF

```
cd vika_docs
sphinx-build -b rinoh source build/rinoh     # build/rinoh/VIKA.pdf
```

A LaTeX backend is also configured: `make latexpdf` produces the same PDF if a full TeX distribution is installed.

8.6 Known operational notes

- `GZ_PARTITION=vika` must be set for any `gz` CLI call from a fresh shell in the container.
- On NVIDIA and Wayland the Gazebo GUI is forced onto software GL to avoid a GLX segfault. The server, controllers and HMI do not depend on the GUI.
- Nodes launched with `docker exec` are not children of the start script, so `. /start-docker.sh stop` explicitly kills them to avoid stale duplicates.

Chapter 9

API Reference

This chapter summarises the ROS 2 packages, the key scripts and the principal interfaces of the VIKA workspace (`vika_ws/src`).

9.1 Packages

Package	Responsibility
`vika_description`	URDF and xacro for both robots, the rail base, the end effectors and the `ros2_control` description, plus meshes, RViz config and xacro unit tests.
`vika_gazebo`	The construction-site world, the brick and pallet models, and helper scripts (`reset_bricks.sh`, `lay_course.sh`, `refill_pallet.sh`).
`vika_moveit`	MoveIt 2 config (SRDF, kinematics, joint limits, controllers) and motion scripts (`tcp_jog.py`, `pick3_lift.py`, `arm_client.py`, `fold_arms.py`), plus config unit tests.
`vika_control`	`ros2_control` controller configuration for both robots.
`vika_bringup`	Top-level launch (`full_demo.launch.py`) and the e2e smoke checks.
`vika_mission`	Mission action definitions (`BuildWall`, `PickBrick`, `PlaceBrick`, `ApplyCement`) and a reference `build_wall.xml` tree. The live mission runs as a small custom Python behaviour-tree engine in `vika_moveit/scripts/bt_node.py`.
`vika_teleop`	Manual teleoperation (joystick and gamepad) config and launch.
`vika_hmi_bridge`	Bridge node exposing HMI commands and state to ROS.
`vika_perception`	Perception package scaffold and launch. The brick detector is the Grounding DINO node `vika_moveit/scripts/dino_detector.py`.
`vika_mcp`	MCP server scaffold exposing world and mission controls to an external LLM.

9.2 Key scripts (`vika_moveit/scripts`)

`tcp_jog.py`
: Console HMI for the required linear TCP jog. Reads `x+/x-/y+/y-/z+/z-` commands, computes inverse kinematics for the offset Cartesian target of the selected arm and executes the trajectory, moving the TCP along a straight line.

`pick3_lift.py`
: Picks a row of three bricks: lower the suction gripper onto the row, attach the `DetachableJoint` grasp (one per dynamic row) and lift.

arm_client.py
Reusable client for inverse kinematics (with seed and collision options), MoveGroup planning and trajectory-action execution.

fold_arms.py
Drives both arms into the compact folded stow pose used at start-up.

hmi_bridge.py
Translates the `/hmi/*` topics the web dashboard publishes into robot motion, driving each robot `arm_controller` and `rail_controller` trajectory actions and the suction attach and detach topics.

bt_node.py
The live mission behaviour-tree engine (Sequence, Fallback, Condition, Action). Listens on `/hmi/mission` (`START` and `STOP`), drives the robots through the `/hmi/*` topics, and publishes the flattened tree on `/bt/state` for the HMI.

dino_detector.py
The optional Grounding DINO brick detector (see AI Modules (Bonus, future work)).

9.3 Principal interfaces

Telemetry and planning:

- `/joint_states`: joint telemetry per robot (published by `joint_state_broadcaster`); drives the HMI digital twin.
- `/move_action` and `/robot_b/move_action`: the MoveIt 2 `MoveGroup` action used for the Cartesian TCP jog.
- `<robot>/arm_controller/follow_joint_trajectory`: arm motion.
- `<robot>/rail_controller/follow_joint_trajectory`: the prismatic rail carriage.
- TF chain `world` to `*_arm_tcp`: TCP pose for planning and HMI readout.

HMI command topics (served by `hmi_bridge.py`):

- `/hmi/active_robot` (`String`): select `robot_a` or `robot_b`.
- `/hmi/cmd` (`String`): `HOME` or `STOP`.
- `/hmi/joint_jog` and `/hmi/joint_set` (`Float64MultiArray`): relative and absolute joint targets.
- `/hmi/rail_jog` and `/hmi/rail_to` (`Float64`): relative and absolute rail.
- `/hmi/tcp_jog` (`Vector3`): Cartesian `dx, dy, dz` linear TCP jog.
- `/hmi/suction` (`Bool`) and `/hmi/suck` (`String`): vacuum gripper.
- `/suction/r0_0/attach` and `/suction/r0_0/detach`: grasp/release the dynamic pick row (the row id is `r`-prefixed because a topic segment may not start with a digit).

Mission and perception:

- `/hmi/mission` (`String`, `START` or `STOP`): drive the build mission.
- `/bt/state` (`String` JSON): the flattened behaviour tree for the HMI.
- `/hmi/detect` (`Empty`): trigger a brick detection.
- `/detect/result` (`String` JSON) and `/detect/image/compressed`: the detector output.

9.4 MCP tools (`vika_mcp`, scaffold)

The server declares `list_models`, `spawn_model` and `mission_start`. The call dispatch is being wired to the `/hmi/*` and Gazebo interfaces.

- genindex
- modindex